

BuildLedger

Construction Contract & Vendor Management System

Microservices Architecture

Architecture Overview

BuildLedger follows a **microservices architecture** where each domain capability is implemented as an independent, deployable service. All client traffic passes through a single **API Gateway** which handles routing, authentication filtering, and rate limiting. Services self-register with a **Eureka Service Registry** enabling dynamic discovery without hardcoded endpoints. Asynchronous communication between services is handled by a **Kafka / RabbitMQ message broker**. Each service owns its own database schema, ensuring loose coupling and independent scaling.

Full Architecture Diagram

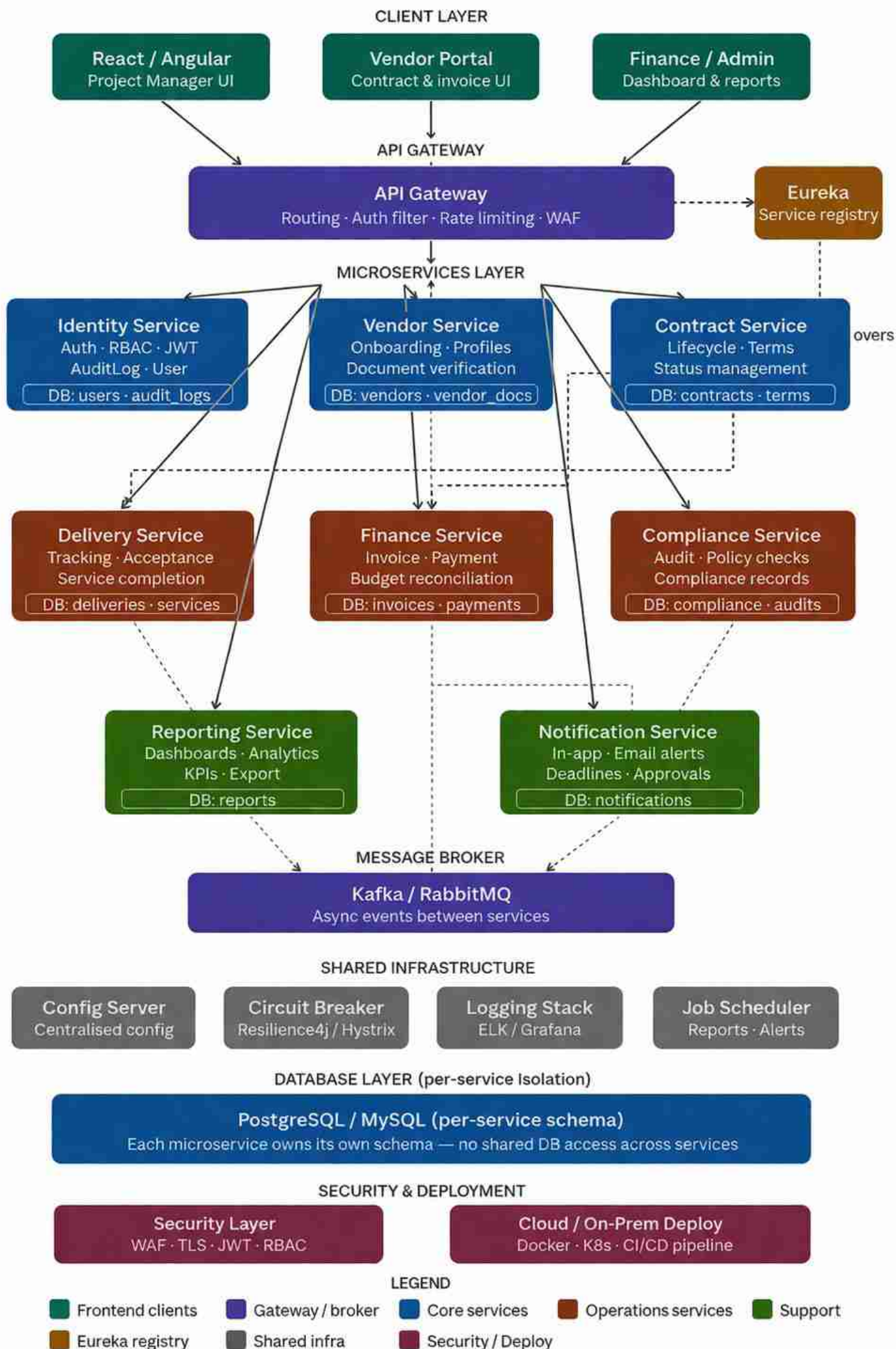


Figure 1 — BuildLedger Microservices Architecture

Microservice Catalogue

The table below summarises each microservice, its responsibilities, domain entities, and database tables.

Service	Responsibilities	Entities	DB Tables
Identity Service	Authentication, RBAC, JWT token management, audit logging	User, AuditLog	users, audit_logs
Vendor Service	Vendor registration, profile management, document verification	Vendor, VendorDocument	vendors, vendor_documents
Contract Service	Contract creation, lifecycle management, terms and conditions	Contract, ContractTerm	contracts, contract_terms
Delivery Service	Delivery tracking, service completion, acceptance workflow	Delivery, Service	deliveries, services
Finance Service	Invoice submission, payment processing, budget reconciliation	Invoice, Payment	invoices, payments
Compliance Service	Policy enforcement, compliance records, audit management	ComplianceRecord, Audit	compliance_records, audits
Reporting Service	Dashboards, KPI analytics, report generation and export	Report	reports
Notification Service	In-app and email alerts for deadlines, approvals, delays	Notification	notifications

Shared Infrastructure Components

Component	Role & Responsibility
API Gateway	Single entry point; routing, auth filter, rate limiting, WAF
Eureka Server	Service registry — all microservices register and discover via Eureka
Config Server	Centralised configuration management for all services
Circuit Breaker	Resilience4j / Hystrix — prevents cascading failures
Message Broker	Kafka / RabbitMQ — async inter-service event streaming
Logging & Monitoring	ELK stack + Grafana; centralised logs and KPI dashboards
Job Scheduler	Quartz / Spring Scheduler — automated reports and deadline alerts

Communication Patterns

Pattern	Used By	Protocol
Synchronous REST	API Gateway → all services	HTTP/HTTPS
Service Discovery	All services ↔ Eureka	Eureka REST API
Async Event Streaming	Delivery / Finance / Compliance → Notification	Kafka / RabbitMQ
Cross-Service Calls	Contract → Delivery, Contract → Finance	REST (internal)
Scheduled Jobs	Reporting Service, Notification Service	Quartz Scheduler

Non-Functional Requirements

Attribute	Requirement
Performance	Handle 5,000 concurrent users across multiple projects
Security	RBAC, TLS encryption, WAF at gateway, immutable audit logs
Scalability	Stateless microservices; horizontal scaling per service
Availability	99.9% uptime; circuit breakers prevent cascading failures
Observability	Centralised logging (ELK), metrics (Grafana), distributed tracing
Maintainability	Independent deployments, API versioning, automated DB migrations

Technology Stack

Layer	Technology
Frontend	React / Angular — responsive dashboards
Backend	Spring Boot (Java) or ASP.NET Core — REST microservices
API Gateway	Spring Cloud Gateway / Kong
Service Registry	Netflix Eureka (Spring Cloud Netflix)
Message Broker	Apache Kafka / RabbitMQ
Database	PostgreSQL / MySQL / SQL Server (per-service schema)
Security	Spring Security, JWT, OAuth2, WAF
Observability	ELK Stack, Prometheus, Grafana
Deployment	Docker, Kubernetes, CI/CD (GitHub Actions / Jenkins)